Part I, Week 01, Day 04 — SVD & PCA Foundations

0) Why Do We Need SVD (Limitations of Eigendecomposition)

In Day 03 you learned **Eigendecomposition**. It is powerful, but it has an important limitation.

Eigendecomposition usually works **directly only for square matrices (n x n)**.

Additionally, not every square matrix has enough independent eigenvectors.
Such matrices are called **defective matrices**, and in those cases the decomposition cannot be performed properly.

However, **real-world datasets are usually rectangular matrices**:

- m samples (data points / rows)
- n features (columns)

So the dataset matrix A usually has the shape:

A : m x n

where **m is often not equal to n**.

To handle this real-world situation, we use **Singular Value Decomposition (SVD)**.
SVD is often called the **"Swiss Army Knife" of linear algebra** because:

- It works for **any real matrix A (m x n)**, whether square or rectangular.
- It decomposes the matrix using **orthogonal bases**, which makes it **numerically stable**.
- Because of this stability and flexibility, it is extremely useful in **machine learning applications**.

1) The SVD Theorem (Core Formula)

Any real matrix A (m x n) can be decomposed as:

A = U * Sigma * V^T

Where:

A : m x n
U : m x m
Sigma : m x n
V^T : n x n

The most important property is:

U and V are orthogonal matrices (in the real case).

Orthogonal means:

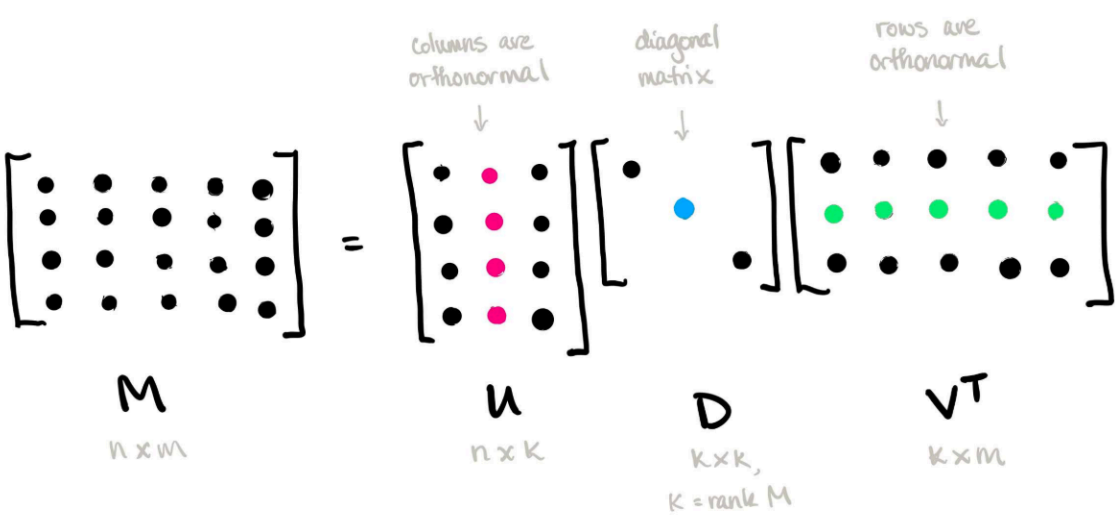
U^T * U = I (m x m identity matrix)

V^T * V = I (n x n identity matrix)

What does this mean geometrically?

The **columns of U and V form orthonormal basis vectors**, meaning:

- They are perpendicular to each other.
- Each vector has unit length.



2) The Anatomy of the Three Matrices

2.1 U — Left Singular Vectors (m x m)

The columns of U are called **left singular vectors**.

Interpretation:

- U provides an **orthonormal basis for the column space (output space)** of A.

Machine Learning intuition:

- U captures the **relationships and structure among the data samples (rows)**.

In simple terms:

When the transformation rotates data from **V-space into U-space**, the matrix **U helps reveal row-wise patterns or relationships between samples**.

2.2 Sigma — Singular Values (m x n diagonal matrix)

Sigma is a **diagonal matrix** (rectangular diagonal).

This means:

The main diagonal contains the singular values:

sigma_1, sigma_2, ..., sigma_r

All other entries are zero.

Properties:

sigma_i >= 0

Sorted order:

sigma_1 >= sigma_2 >= ... >= 0

Interpretation:

Each singular value **sigma_i measures the strength or importance of its corresponding component**.

Large sigma values indicate:

- More **data variation**
- More **energy or information** in that direction.

Machine Learning context:

In PCA, we often:

- Keep the **largest singular values**
- Drop the smaller ones

This leads to **dimensionality reduction**.

2.3 V^T — Right Singular Vectors (n x n)

The rows of V^T (or equivalently, the columns of V) are called **right singular vectors**.

Interpretation:

- V provides an **orthonormal basis for the feature space (input space)**.

Machine Learning intuition:

- The vectors in V represent **feature combinations or principal directions**.

In PCA terminology:

The **first few columns of V correspond to the principal component directions**.

In simpler words:

V tells us **which combinations of features create the most meaningful variation in the data**.

3) Interpreting SVD as “Rotate + Scale + Rotate”

The decomposition

A = U * Sigma * V^T

can be understood geometrically as three sequential operations.

Step 1: Apply V^T (Rotate or Reflect)

This rotates the coordinate system of the **input feature space**, aligning it with the true axes of variance.

Step 2: Apply Sigma (Scale)

Sigma stretches or compresses the data along these new axes.
Each singular value controls how much scaling occurs along its axis.

Step 3: Apply U (Rotate or Reflect again)

Finally, U rotates the transformed vectors into their **final orientation in the output space**.

So overall, matrix A performs a linear transformation that:

1. Changes the coordinate system (V^T)
2. Scales magnitudes along new axes (Sigma)
3. Rotates the result into the final output orientation (U)

4) Why SVD Is the Foundation of PCA

The goal of **Principal Component Analysis (PCA)** is to find directions where the **data variance is maximized**.

Higher variance usually means **more informative directions in the dataset**.

In SVD:

- The **columns of V provide the principal directions (feature combinations)**.
- The **singular values sigma_i measure the importance of each direction**.

Therefore, PCA can be implemented by:

1. Keeping the **top k singular values (sigma_1 ... sigma_k)**
2. Keeping the corresponding **columns of V**

The dataset can then be projected onto these directions to obtain a **lower-dimensional representation**.

5) Quick Shape Check (Document-Safe)

Assume:

A is an m x n matrix.

Then:

U : m x m
Sigma : m x n
V^T : n x n

Matrix multiplication consistency:

U (m x m) * Sigma (m x n) = (m x n)

(m x n) * V^T (n x n) = (m x n)

Therefore:

U * Sigma * V^T = A

2) Geometric Intuition: “Rotate, Scale, Rotate” (SVD Geometry)

SVD says:

A = U * Sigma * V^T

This is not just an algebraic factorization; it has a very clear geometric meaning.

Any linear transformation A: R^n -> R^m (mapping an n-dimensional input vector to an m-dimensional output vector) can always be broken into three simple sequential geometric operations:

1. V^T : Rotate/Reflect (in the input space)
2. Sigma : Scale (stretch/squish)
3. U : Rotate/Reflect (in the output space)

In simple words:

First rotate, then stretch/compress, then rotate again.

What do “Rotate/Reflect” and “Scale” mean?

Rotate (Rotation)

- Does not change the vector length
- Only changes the direction
- Preserves angles

Reflect (Reflection)

- Flips the vector like a mirror (across some axis)
- Does not change the vector length

We say “Rotate/Reflect” together because an orthogonal matrix (like U or V) can represent either a rotation or a reflection.

Orthogonal matrix property (doc-safe):

Q^T * Q = I

Here Q can be U or V.

Scale (Scaling)

- Makes some components larger and others smaller
- Usually changes the vector length

Step 1) V^T (Rotate/Reflect): “Align the input space with the true axes”

Let the original input vector be x (n x 1).

First we apply:

x' = V^T * x

Meaning:

- We are changing the coordinate system in the input space.
- We rotate the data so that it aligns with the “true axes of variance.”

Intuition:

- In feature space, the strongest variance (information) is not always aligned with the standard axes (like the x-axis and y-axis).
- Often the strongest variance lies along a diagonal direction.
- V^T rotates the space so that these diagonal directions become the new coordinate axes.

PCA link:

- The columns of V are the principal directions (feature combinations).
- Applying V^T means rotating/projecting into the “principal coordinate system.”

Step 2) Sigma (Scale): “Stretch/squish along the new axes”

Next we apply:

z = Sigma * x'

Sigma is a diagonal-ish (rectangular diagonal) matrix whose diagonal entries are the singular values:

Sigma diagonal entries: sigma_1, sigma_2, ..., sigma_r

with:

sigma_1 >= sigma_2 >= ... >= 0

Scaling intuition:

- Each axis gets its own scaling factor:
    - 1st axis is scaled by sigma_1
    - 2nd axis is scaled by sigma_2
    - ...
- If sigma is large, the vector is stretched more along that direction.
- If sigma is small, the vector is stretched less (or effectively compressed) along that direction.
- If sigma = 0, the information in that direction collapses (it gets flattened).

Machine Learning intuition:

- Large sigma means the component is more important (higher data energy/variance).
- In PCA, keeping the top-k singular values preserves most of the information.

Step 3) U (Rotate/Reflect): “Place the result into the final output orientation”

Finally we apply:

y = U * z

Meaning:

- We map the scaled vector into the output space (R^m) and set its final orientation.
- This operation preserves lengths and angles (it rotates/reflects), but changes direction/placement.

Intuition:

- After Step 2, the vector has been scaled correctly along the right axes, but it is not yet placed in the final coordinate frame of the output space.
- U performs that final rotation/reflection into the output space.

Putting it all together: “Why Rotate-Scale-Rotate works for any A”

A general linear transformation A can simultaneously:

- rotate
- stretch/compress
- mix dimensions

SVD guarantees we can always decompose that complex action as:

1. Use V^T to rotate the input into a basis where the transformation becomes simple (axis-aligned).
2. Use Sigma to do pure scaling along each axis (the simplest kind of linear transform).
3. Use U to rotate the result into its final output orientation.

Mental picture (2D intuition)

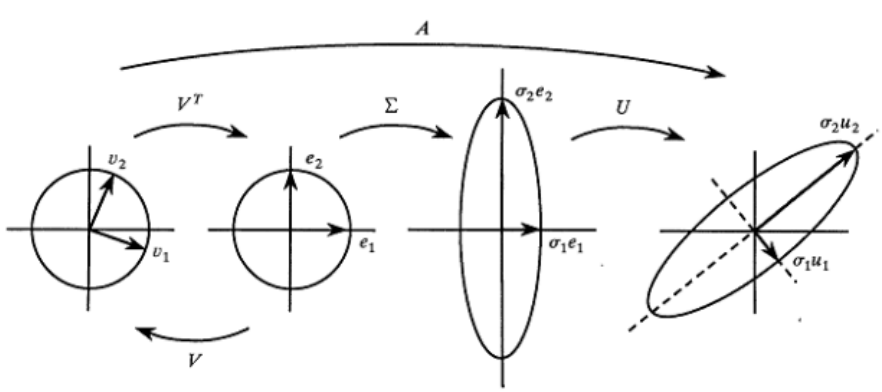
Imagine a unit circle in the input space (2D case).

After applying A:

- The circle usually becomes an ellipse (due to stretching/compressing and rotating).

SVD interpretation:

- V^T: rotate the circle so it aligns with the ellipse’s axes
- Sigma: stretch the circle into an ellipse (major/minor axis lengths are the singular values)
- U: rotate the ellipse into its final position



3) The Bridge to PCA — Why SVD Gives PCA “for Free”

1) What PCA Is Trying to Do

PCA (Principal Component Analysis) aims to:

- Find a set of directions in the data along which the variance (spread / information) is maximal.
- These directions are called the **principal components**.
- Then we keep only the top-k directions to reduce dimensionality (**dimensionality reduction**).

2) The Classic PCA Route: Covariance Matrix + Eigenvectors

Assume your mean-centered data matrix is X (m x n), where:

- m = number of samples
- n = number of features
- mean-centered means: the mean of each feature column has been subtracted, so each column has mean 0.

PCA typically builds the covariance matrix:

C = (1/(m-1)) * X^T * X

Shapes:

- X : (m x n)
- X^T : (n x m)
- X^T * X : (n x n) -> square matrix
- C : (n x n)

Then PCA proceeds by:

- computing the eigenvectors of C
- the top eigenvectors correspond to the principal component directions

3) What’s the Problem: Why X^T * X Can Be Numerically Unstable

Computing X^T * X directly can cause numerical issues:

- For large datasets, numerical errors increase.
- The condition number can become worse, making inversion/eigendecomposition harder.
- Floating-point rounding errors can be amplified.

That is why, in practice, the “C = X^T X” approach can be numerically shaky.

4) The Stable Route: Compute SVD of X Directly

Instead, we compute SVD directly on the raw mean-centered dataset X:

X = U * Sigma * V^T

Shapes:

- U : (m x m)
- Sigma : (m x n)
- V^T : (n x n)

Key idea:

- Instead of explicitly forming X^T X, we compute SVD of X first.

5) Substitute SVD into the Covariance Equation

Ignoring the constant factor (1/(m-1)) for the moment:

C ∝ X^T * X

Substitute X = U * Sigma * V^T:

C ∝ (U * Sigma * V^T)^T * (U * Sigma * V^T)

Transpose rule:
(ABC)^T = C^T * B^T * A^T

So:

(U * Sigma * V^T)^T = V * Sigma^T * U^T

Now multiply:

C ∝ (V * Sigma^T * U^T) * (U * Sigma * V^T)

Re-group terms:

$C \propto V * \text{Sigma}^T * (U^T * U) * \text{Sigma} * V^T$

Since U is orthogonal:

$U^T * U = I$

Therefore:

$C \propto V * \text{Sigma}^T * \text{Sigma} * V^T$

Now, $\text{Sigma}^T * \text{Sigma}$ is a diagonal matrix whose diagonal entries are:

$\text{sigma}_1^2, \text{sigma}_2^2, ..., \text{sigma}_r^2$

So we can write:

$C \propto V * (\text{Sigma}^2) * V^T$

Here “Sigma^2” means the diagonal matrix produced by $\text{Sigma}^T * \text{Sigma}$ (i.e., singular values squared).

6) This Matches the Eigendecomposition Form Exactly

The eigendecomposition form is:

$C = Q * \text{Lambda} * Q^T$

Where:

- Q = matrix of eigenvectors
- Lambda = diagonal matrix of eigenvalues

Our derived form is:

$C \propto V * (\text{Sigma}^T \text{Sigma}) * V^T$

Matching terms:

- Q corresponds to V
- Lambda corresponds to $(\text{Sigma}^T \text{Sigma})$, scaled by $(1/(m-1))$

Conclusion:

- The columns of V (right singular vectors) are exactly the PCA principal component directions.

7) Relationship Between Eigenvalues and Singular Values

Original covariance:

$C = (1/(m-1)) * X^T * X$

And we have:

$X^T * X = V * (\Sigma^T \Sigma) * V^T$

So:

$C = (1/(m-1)) * V * (\Sigma^T \Sigma) * V^T$

Therefore each eigenvalue is:

$\lambda_i = (\sigma_i^2) / (m-1)$

Doc-safe summary:

- principal component directions = columns of V
- eigenvalues = (singular values squared) / (m-1)

8) What “SVD Gives PCA for Free” Means

To perform PCA you need:

- principal directions
- explained variance (eigenvalues)

From SVD you directly get:

- directions: V
- strengths: σ_i, and variance is $\sigma_i^2/(m-1)$

So PCA can be done **without explicitly forming the covariance matrix**.

4) Low-Rank Approximation: Data Compression (Truncated SVD)

1) Core Idea: Not All Information Is Equally Important

In SVD we write:

$A = U * \Sigma * V^T$

The diagonal of Sigma contains the singular values:

$\sigma_1 >= \sigma_2 >= ... >= \sigma_r >= 0$

This sorting (largest to smallest) is crucial because, in many real datasets:

- Large singular values capture most of the meaningful signal/structure in the data.
- Small trailing singular values often behave like noise or irrelevant detail.

So, keeping only the first few singular values often preserves the main patterns of the dataset.

2) What Truncated SVD Means

Truncated SVD means:

- Keep only the top-k singular values.
- Set the remaining singular values to 0.

Here k is a small number such that:

$k << r$

where r is rank(A) or the maximum possible rank.

3) Low-Rank Approximation Formula (Doc-Safe)

The approximation created by keeping only the top-k components is:

$A \approx A_k = U_k * \Sigma_k * V_k^T$

Matrix sizes:

- $A : m \times n$

- U_k : m x k (the first k columns of U)
- Sigma_k : k x k (diagonal matrix of the top-k singular values)
- V_k^T : k x n (the first k rows of V^T)
  (equivalently, V_k : n x k)

Multiplication check:

(U_k : m x k) * (Sigma_k : k x k) = (m x k)
(m x k) * (V_k^T : k x n) = (m x n) = A_k

4) Why Low-Rank Means “Compression”

To store the full matrix A, you need:

m * n values

To store the low-rank approximation A_k, you need approximately:

- U_k : m * k values
- Sigma_k : k values (if storing only the diagonal) or k * k (if storing full)
- V_k^T : k * n values

Total storage is roughly:

m*k* + *k* + *k*n (or m*k* + *k*k + k*n)

When k is small:

m*k* + *k*n << m*n

So memory usage is much lower, and computation can also become significantly faster.

5) “Signal vs Noise” Intuition (Why We Can Drop Small Singular Values)

Singular values (sigma_i) effectively tell you:

How strongly the i-th component explains the data.

If sigma_i is very small:

- Variation in that direction is tiny.
- It often corresponds to measurement noise, background detail, or weak patterns.

So dropping small singular values typically:

- reduces noise
- preserves the dominant structure of the data

6) Engineering Applications (Common ML Use Cases)

6.1 Image Compression

A grayscale image can be represented as a matrix:

- rows = pixel rows
- cols = pixel columns
- entry = intensity value

If we apply truncated SVD:

- large-scale structure (edges, major shapes) is preserved
- small details/noise are reduced
- the image often looks nearly the same while requiring much less storage

6.2 NLP: Latent Semantic Analysis (LSA)

In NLP, you often build a term-document matrix:

- rows = words/terms
- cols = documents
- entry = frequency/weight (e.g., TF-IDF)

Applying SVD and keeping top-k components:

- maps words and documents into a lower-dimensional “latent topic” space
- helps handle synonymy and polysemy
- improves search and retrieval quality

6.3 Recommendation Systems: Matrix Factorization (Netflix-Style)

In recommenders, you often have a ratings matrix:

- rows = users
- cols = items/movies
- entry = rating (many entries are missing)

Low-rank assumption:
User preferences can be explained using a small number of latent factors
(e.g., action vs romance preference, comedy taste, etc.)

From truncated SVD / matrix factorization you get:

- user embeddings (from U_k side)
- item embeddings (from V_k side)
- strength/scales (from Sigma_k)

This is the mathematical foundation behind Netflix-style recommendation systems.

7) What “Rank” Means

Rank roughly means:

- the number of independent directions/components in a matrix.

Low-rank approximation means:

- instead of using all components, we approximate the matrix using fewer components, while staying as accurate as possible.

Practical viewpoint:

- low rank often implies the data is driven by a small number of underlying factors.